

CSA0624 - Design and Analysis of Algorithms

Lab Practical Manual — Code, Output & Complexity Analysis

Name: Aryan Arun

Reg.No: 192411157

15 Experiments: Searching · Sorting · Recursion · Divide & Conquer

Experiment 1: Linear Search (Iterative)

Objective: To analyze the time complexity of linear search.

Problem Statement: Write a Python program to search for a key in an array using linear search.

Python Code

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 50]
key = 30
result = linear_search(arr, key)
print(f"Array: {arr}, Key: {key}")
print(f"Key found at index {result}" if result != -1 else "Key not found")
```

Output

```
Array: [10, 25, 30, 45, 50], Key: 30
Key found at index 2
```

Complexity: Ω (Best): $O(1)$ — key is the first element. O (Worst): $O(n)$ — key is last or absent. Θ (Average): $\Theta(n)$.

Explanation: Demonstrates sequential scanning: each element is checked one by one from the start, making the algorithm simple but inefficient for large inputs since it gains no benefit from any existing order in the data.

Experiment 2: Binary Search (Recursive)

Objective: To compare the complexity of recursive binary search against linear search.

Problem Statement: Implement recursive binary search in Python.

Python Code

```
def binary_search_recursive(arr, key, low, high):
    if low > high:
        return -1
    mid = low + (high - low) // 2
    if arr[mid] == key:
        return mid
    elif arr[mid] < key:
        return binary_search_recursive(arr, key, mid + 1, high)
    else:
        return binary_search_recursive(arr, key, low, mid - 1)

arr = [5, 10, 15, 20, 25]
key = 20
result = binary_search_recursive(arr, key, 0, len(arr) - 1)
print(f"Array: {arr}, Key: {key}")
print(f"Key found at index {result}" if result != -1 else "Key not found")
```

Output

```
Array: [5, 10, 15, 20, 25], Key: 20
Key found at index 3
```

Explanation: Shows the divide-and-conquer principle: the search space is halved at every recursive call, giving logarithmic efficiency that scales far better than linear search on large sorted datasets.

Experiment 3: Factorial (Iterative vs Recursive)

Objective: To compare iterative and recursive factorial computation.

Problem Statement: Write programs to compute factorial using both methods.

Python Code

```
def factorial_iterative(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

n = 5
print(f"n = {n}")
print(f"Factorial (Iterative): {factorial_iterative(n)}")
print(f"Factorial (Recursive): {factorial_recursive(n)}")
```

Output

```
n = 5
Factorial (Iterative): 120
Factorial (Recursive): 120
```

Complexity: Both approaches run in $O(n)$ time, since each performs exactly n multiplications.

Explanation: Highlights the trade-off between recursion depth and iterative loops: the recursive version uses $O(n)$ call-stack space in addition to $O(n)$ time, while the iterative version uses only $O(1)$ extra space, making it more memory-efficient for large n .

Experiment 4: Fibonacci Series

Objective: To analyze recursive vs iterative Fibonacci generation.

Problem Statement: Generate the first n Fibonacci numbers.

Python Code

```
def fibonacci_iterative(n):
    fib = [0, 1]
    for i in range(2, n):
        fib.append(fib[-1] + fib[-2])
    return fib[:n]

def fibonacci_recursive_naive(n):
    if n <= 1:
        return n
    return fibonacci_recursive_naive(n - 1) + fibonacci_recursive_naive(n - 2)

def fibonacci_memoized(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci_memoized(n - 1, memo) + fibonacci_memoized(n - 2, memo)
    return memo[n]

n = 6
print(f"n = {n}")
print(f"Fibonacci (Iterative): {fibonacci_iterative(n)}")
print(f"Fibonacci (Naive Recursive): {[fibonacci_recursive_naive(i) for i in range(n)]}")
print(f"Fibonacci (Memoized): {[fibonacci_memoized(i) for i in range(n)]}")
```

Output

```
n = 6
Fibonacci (Iterative): [0, 1, 1, 2, 3, 5]
Fibonacci (Naive Recursive): [0, 1, 1, 2, 3, 5]
Fibonacci (Memoized): [0, 1, 1, 2, 3, 5]
```

Complexity: Iterative: $O(n)$. Naïve recursive: $O(2^n)$ (exponential, due to repeated re-computation of the same sub-problems). Recursive with memoization: $O(n)$ (each sub-problem solved once and cached).

Explanation: Illustrates how the naïve recursive approach recomputes the same Fibonacci values many times over, leading to exponential blow-up, while memoization (caching previously computed results) or a simple iterative loop brings this down to linear time.

Experiment 5: Matrix Multiplication

Objective: To analyze nested-loop complexity.

Problem Statement: Multiply two matrices using loops.

Python Code

```
def matrix_multiply(A, B):
    rows_A, cols_A = len(A), len(A[0])
    cols_B = len(B[0])
    result = [[0] * cols_B for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]
    return result

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
result = matrix_multiply(A, B)
print(f"Matrix A: {A}")
print(f"Matrix B: {B}")
print(f"Product A x B: {result}")
```

Output

```
Matrix A: [[1, 2], [3, 4]]
Matrix B: [[5, 6], [7, 8]]
Product A x B: [[19, 22], [43, 50]]
```

Complexity: $O(n^3)$ for two $n \times n$ matrices, due to three nested loops (rows, columns, and the summation index).

Explanation: Shows cubic growth: for an $n \times n$ matrix, the algorithm performs n^3 multiply-add operations, since for each of the n^2 output cells, n multiplications must be summed — this is the classical bound for naive matrix multiplication.

Experiment 6: Merge Sort

Objective: To analyze divide-and-conquer sorting.

Problem Statement: Implement merge sort.

Python Code

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = [38, 27, 43, 3, 9, 82, 10]
print(f"Input: {arr}")
print(f"Sorted: {merge_sort(arr)}")
```

Output

```
Input: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]
```

Complexity: Always $O(n \log n)$ — for best, average, and worst case alike.

Explanation: Demonstrates the guaranteed efficiency of divide-and-conquer: the array is repeatedly split in half ($\log n$ levels), and merging two sorted halves takes $O(n)$ at each level, giving $O(n \log n)$ regardless of the input's initial order.

Experiment 7: Quick Sort

Objective: To analyze recursive quick sort.

Problem Statement: Implement quick sort.

Python Code

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

arr = [10, 7, 8, 9, 1, 5]
print(f"Input: {arr}")
print(f"Sorted: {quick_sort(arr)}")
```

Output

```
Input: [10, 7, 8, 9, 1, 5]
Sorted: [1, 5, 7, 8, 9, 10]
```

Complexity: Ω (Best): $O(n \log n)$ — balanced partitions. O (Worst): $O(n^2)$ — highly unbalanced partitions (e.g., already-sorted input with a poor pivot choice). Θ (Average): $\Theta(n \log n)$.

Experiment 8: Tower of Hanoi

Objective: To analyze exponential recursion.

Problem Statement: Solve the Tower of Hanoi puzzle for n disks.

Python Code

```
def tower_of_hanoi(n, source, auxiliary, target, moves):
    if n == 1:
        moves.append(f"Move disk 1 from {source} to {target}")
        return
    tower_of_hanoi(n - 1, source, target, auxiliary, moves)
    moves.append(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n - 1, auxiliary, source, target, moves)

n = 3
moves = []
tower_of_hanoi(n, 'A', 'B', 'C', moves)
print(f"n = {n}")
for m in moves:
    print(m)
print(f"Total moves: {len(moves)}")
```

Output

```
n = 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7
```

Complexity: $O(2^n)$ — the recurrence $T(n) = 2T(n-1) + 1$ solves to $2^n - 1$ total moves.

Explanation: Illustrates exponential growth in recursive calls: each additional disk doubles the number of moves required (plus one), so the problem quickly becomes intractable for large n even though the algorithm itself is simple and elegant.

Experiment 9: GCD (Euclidean Algorithm)

Objective: To analyze recursive GCD computation.

Problem Statement: Implement the Euclidean algorithm for GCD.

Python Code

```
def gcd_recursive(a, b):  
    if b == 0:  
        return a  
    return gcd_recursive(b, a % b)  
  
a, b = 48, 18  
print(f"a = {a}, b = {b}")  
print(f"GCD: {gcd_recursive(a, b)}")
```

Output

```
a = 48, b = 18  
GCD: 6
```

Complexity: $O(\log(\min(a, b)))$ — the remainder shrinks geometrically with each recursive call (related to the Fibonacci ratio in the worst case).

Explanation: Shows an efficient recursive algorithm: unlike the exponential recursion of Tower of Hanoi, each step of the Euclidean algorithm reduces the problem size multiplicatively (by taking a remainder), giving logarithmic — not linear or exponential — complexity.

Experiment 10: Insertion Sort

Objective: To analyze iterative sorting and input sensitivity.

Problem Statement: Implement insertion sort.

Python Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

arr = [12, 11, 13, 5, 6]
print(f"Input: {arr}")
print(f"Sorted: {insertion_sort(arr.copy())}")
```

Output

```
Input: [12, 11, 13, 5, 6]
Sorted: [5, 6, 11, 12, 13]
```

Complexity: Ω (Best): $O(n)$ — array is already sorted, inner while-loop never shifts. O (Worst): $O(n^2)$ — array is reverse sorted, maximal shifting. Θ (Average): $\Theta(n^2)$.

Explanation: Demonstrates input sensitivity: Insertion Sort's running time depends heavily on how out-of-order the input is — nearly-sorted data is handled in near-linear time, while reverse-sorted data forces the maximum number of shifts.

Experiment 11: Heap Sort

Objective: To analyze heap-based sorting.

Problem Statement: Implement heap sort.

Python Code

```
def heapify(arr, n, i):
    largest = i
    left, right = 2 * i + 1, 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr

arr = [4, 10, 3, 5, 1]
print(f"Input: {arr}")
print(f"Sorted: {heap_sort(arr.copy())}")
```

Output

```
Input: [4, 10, 3, 5, 1]
Sorted: [1, 3, 4, 5, 10]
```

Complexity: Always $O(n \log n)$ — building the heap takes $O(n)$, and each of the n extraction steps takes $O(\log n)$ to re-heapify.

Explanation: Shows consistent performance regardless of input arrangement: because the algorithm relies on the heap's structural properties rather than the data's initial order, Heap Sort guarantees $O(n \log n)$ in the best, average, and worst cases alike, unlike Quick Sort.

Experiment 12: Counting Inversions

Objective: To analyze a modified merge sort for counting inversions.

Problem Statement: Count the number of inversions in an array (pairs where $i < j$ but $\text{arr}[i] > \text{arr}[j]$).

Python Code

```
def count_inversions(arr):
    if len(arr) <= 1:
        return arr, 0
    mid = len(arr) // 2
    left, inv_left = count_inversions(arr[:mid])
    right, inv_right = count_inversions(arr[mid:])
    merged, inv_split = merge_and_count(left, right)
    return merged, inv_left + inv_right + inv_split

def merge_and_count(left, right):
    result = []
    i = j = 0
    inversions = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
            inversions += len(left) - i  # all remaining left elements form inversions
    result.extend(left[i:])
    result.extend(right[j:])
    return result, inversions

arr = [2, 4, 1, 3, 5]
sorted_arr, inv_count = count_inversions(arr)
print(f"Input: {arr}")
print(f"Number of inversions: {inv_count}")
```

Output

```
Input: [2, 4, 1, 3, 5]
Number of inversions: 3
```

Complexity: $O(n \log n)$ — same recurrence structure as merge sort, since counting is folded into the existing merge step at no extra asymptotic cost.

Explanation: Shows divide-and-conquer applied beyond plain sorting: whenever an element from the right half is placed before remaining elements of the left half during the merge step, all of those remaining left elements form inversions with it, letting the count be computed in the same $O(n \log n)$ pass that sorts the array.

Experiment 13: Maximum Subarray Sum

Objective: To compare Kadane's algorithm vs the divide-and-conquer approach.

Problem Statement: Find the maximum sum of any contiguous subarray.

Python Code

```
def max_subarray_kadane(arr):
    max_so_far = arr[0]
    max_ending_here = arr[0]
    for i in range(1, len(arr)):
        max_ending_here = max(arr[i], max_ending_here + arr[i])
        max_so_far = max(max_so_far, max_ending_here)
    return max_so_far

def max_crossing_sum(arr, low, mid, high):
    left_sum, total = float('-inf'), 0
    for i in range(mid, low - 1, -1):
        total += arr[i]
        left_sum = max(left_sum, total)
    right_sum, total = float('-inf'), 0
    for i in range(mid + 1, high + 1):
        total += arr[i]
        right_sum = max(right_sum, total)
    return left_sum + right_sum

def max_subarray_dc(arr, low, high):
    if low == high:
        return arr[low]
    mid = (low + high) // 2
    left_max = max_subarray_dc(arr, low, mid)
    right_max = max_subarray_dc(arr, mid + 1, high)
    cross_max = max_crossing_sum(arr, low, mid, high)
    return max(left_max, right_max, cross_max)

arr = [-2, -3, 4, -1, -2, 1, 5, -3]
print(f"Input: {arr}")
print(f"Maximum Subarray Sum (Kadane's): {max_subarray_kadane(arr)}")
print(f"Maximum Subarray Sum (Divide & Conquer): {max_subarray_dc(arr, 0, len(arr) - 1)}")
```

Output

```
Input: [-2, -3, 4, -1, -2, 1, 5, -3]
Maximum Subarray Sum (Kadane's): 7
Maximum Subarray Sum (Divide & Conquer): 7
```

Complexity: Kadane's algorithm: $O(n)$ — single linear pass. Divide & Conquer: $O(n \log n)$ — the recurrence $T(n) = 2T(n/2) + O(n)$ for the cross-sum check at each level.

Explanation: Highlights the efficiency gap between a well-designed linear-scan algorithm (Kadane's) and a divide-and-conquer approach that, while asymptotically slower here, illustrates how the same problem can be solved by combining solutions to sub-problems plus a 'crossing' case — a pattern reusable for other range-based problems.

Experiment 14: Exponentiation

Objective: To compare iterative vs recursive fast power computation.

Problem Statement: Compute x^n .

Python Code

```
def power_iterative(x, n):
    result = 1
    for _ in range(n):
        result *= x
    return result

def power_recursive_fast(x, n):
    if n == 0:
        return 1
    half = power_recursive_fast(x, n // 2)
    if n % 2 == 0:
        return half * half
    else:
        return half * half * x

x, n = 2, 10
print(f"x = {x}, n = {n}")
print(f"Power (Iterative): {power_iterative(x, n)}")
print(f"Power (Recursive Fast): {power_recursive_fast(x, n)}")
```

Output

```
x = 2, n = 10
Power (Iterative): 1024
Power (Recursive Fast): 1024
```

Complexity: Iterative: $O(n)$ — one multiplication per loop iteration. Recursive fast power (exponentiation by squaring): $O(\log n)$ — the exponent is halved at every recursive call.

Explanation: Shows how recursion (combined with the squaring identity $x^n = (x^{n/2})^2$) reduces the number of multiplications from linear to logarithmic in the exponent, a technique widely used in fast modular exponentiation for cryptography.

Experiment 15: Closest Pair of Points

Objective: To analyze brute force vs divide-and-conquer for the closest pair problem.

Problem Statement: Find the closest pair of points in a 2D plane.

Python Code

```
import math

def closest_pair_brute_force(points):
    n = len(points)
    min_dist = float('inf')
    closest = None
    for i in range(n):
        for j in range(i + 1, n):
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            dist = math.sqrt(dx * dx + dy * dy)
            if dist < min_dist:
                min_dist = dist
                closest = (points[i], points[j])
    return closest, min_dist

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
closest, dist = closest_pair_brute_force(points)
print(f"Points: {points}")
print(f"Closest pair: {closest[0]}-{closest[1]}, Distance = {dist:.4f} (= sqrt(5))")
```

Output

```
Points: [(1, 2), (4, 5), (7, 8), (3, 1)]
Closest pair: (1, 2)-(3, 1), Distance = 2.2361 (= sqrt(5))
```

Complexity: Brute force: $O(n^2)$ — every pair of points is checked. Optimized (divide-and-conquer): $O(n \log n)$ — points are split by a vertical line and only a narrow 'strip' of candidates near the boundary is rechecked.

Explanation: Demonstrates the quadratic cost of exhaustively checking all point pairs and motivates the divide-and-conquer optimization, which recursively finds the closest pair in each half and then checks only a limited number of candidates that fall within the minimum distance of the dividing line — reducing the complexity from $O(n^2)$ to $O(n \log n)$.